AURA-UDP implementation progress report

Implementation assessment • Approximately 70–75% complete
Deadline-aware reliable transport for critical campus IoT messages

Purvi Valmeti • 24BCE2191
M Harshita Sri • 24BCE2471

GitHub repository: https://github.com/purvi126/AURA-UDP

This report explains what has been built, how it was checked, what the measured results show, and what remains for the final review. The 70–75% figure is an assessment of project readiness, not a measured software or network metric.

The project in one minute

A smart campus sends routine readings and urgent alerts to a monitoring room. A lost or late routine reading may be tolerable; a smoke alert is useful only if it arrives before its deadline. AURA-UDP asks whether a sender can improve timely alert delivery while controlling the cost of retransmissions.

Two tools show different parts of that question. NS2 runs the actual transport experiment and produces timing, loss, and retry measurements. Cisco Packet Tracer is an editable campus network model that shows routers, HTTP connectivity, an IoT gateway, and registered sensor devices. The tools do not exchange live packets.

What is implemented at this milestone

Workstream	Implemented and evidenced	Status
Transport in NS2	Baseline, Selective Repeat, and AURA sender/receiver behavior; ACKs, deadlines, retry and congestion-window decisions.	Built
Evaluation	Three scenarios, paired seed sweeps, policy ablations, fresh-seed checks, metrics, charts, and a local dashboard.	Built
Campus in Packet Tracer	Two routers, monitoring server, native temperature and smoke devices, gateway registration, routed HTTP check.	Built
Final research claim and demo	Wider workloads, stronger fairness/congestion tests, improved retry estimate, and final presentation evidence.	Remaining

How the pieces fit

Campus alert idea → NS2 sender and lossy link → event logs and metrics → comparison charts and dashboard

Campus layout idea → Packet Tracer routers, server, gateway, and sensors → connectivity and registration screenshots

What happens inside NS2

The script ns2/run.tcl builds a sender, router, monitoring receiver, and competing CBR sender. Its 128 Kb/s bottleneck has a short queue and configurable packet loss. Each run creates 30 messages; every fourth is critical and has an absolute deadline. Data and ACK paths have separate loss streams.

Mode	Sender behavior	Purpose
Baseline UDP	Sends a message once; no ACK or retry.	Reference for low overhead.
Fixed Selective Repeat	Uses a four-message window, ACKs and individual retries with a fixed timer.	Reference reliable method.
AURA	Orders urgent alerts by deadline, gives routine traffic regular turns, adapts timer and small window, and checks whether a retry can still arrive in time.	Tests the deadline-aware policy.

The decision being tested

For a critical message, the sender compares estimated forward delivery time with time remaining until its absolute deadline. If the estimated trip is longer, AURA abandons that retransmission. The first send is still attempted while the alert is live. This can save traffic, but a rough estimate may reject a retry that would have succeeded.

Retry when estimated delivery time ≤ deadline − current time

AURA takes RTT samples only from packets that have not been retransmitted, then uses smoothed RTT and variation to set a bounded retry timer. Its sender window starts at four messages, can grow to six after ACK progress, and falls back on a timeout. After three urgent admissions, a waiting routine message gets the next available slot. These are explicit lab rules, not a claim of a complete Internet congestion controller.

What is measured

- Deadline delivery ratio (DDR): critical messages first delivered on time divided by all generated critical messages, including lost and abandoned ones.
- Packet delivery ratio (PDR): unique messages delivered eventually divided by all generated messages. A late alert can count in PDR but not DDR.
- Also recorded: delay for delivered messages, extra sends per generated message, routine delivery, and received background traffic.

NS2 writes events.csv, control.csv, background.csv, a packet trace, and a NAM trace. Python analysis deduplicates message IDs and produces summary CSVs, plots, and the local dashboard. The dashboard reads saved results; it does not run NS2.

Experiment design

The development set uses 30 seed indices across three conditions and three transport modes (270 runs), plus four AURA variants (360 runs). A fresh 30-seed set repeats both sweeps. Total recorded evaluation: 1,260 NS2 runs. Within a scenario, modes use the same topology, generation schedule, and seed index; changed retry behavior means packet-by-packet loss draws are not identical.

NS2 evidence and measured findings

The figure below is generated from the checked-in NS2 run summaries. It compares delivery and retry behavior across moderate loss, tight deadlines, and a higher message rate. It is experimental output, not Packet Tracer traffic.

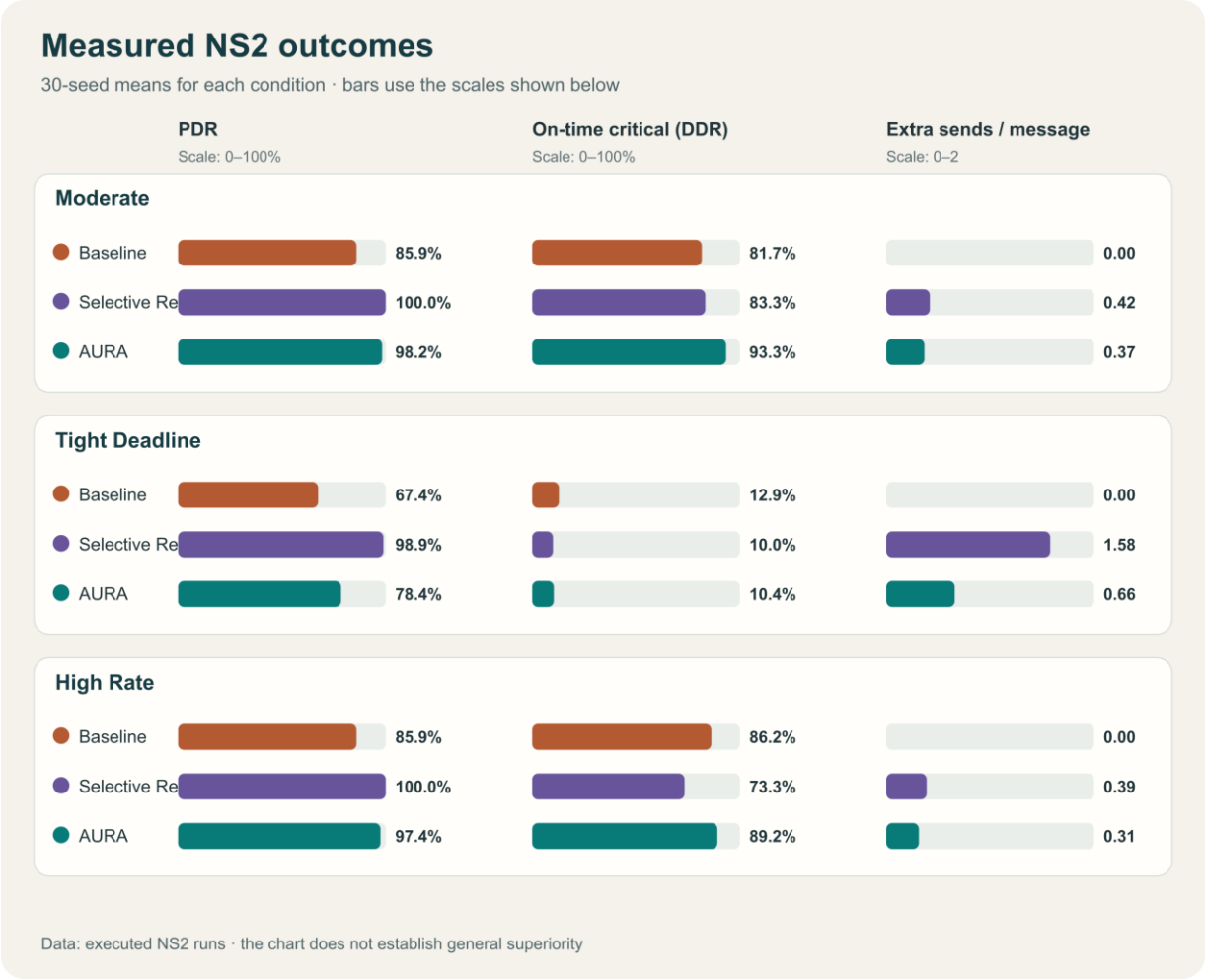


Figure 1 NS2 30-seed comparison from experiments/plots/multiseed.svg

Scenario	Baseline DDR	Fixed SR DDR	AURA DDR
Moderate loss	81.7%	83.3%	93.3%
Tight deadline	12.9%	10.0%	10.4%
High message rate	86.3%	73.3%	89.2%

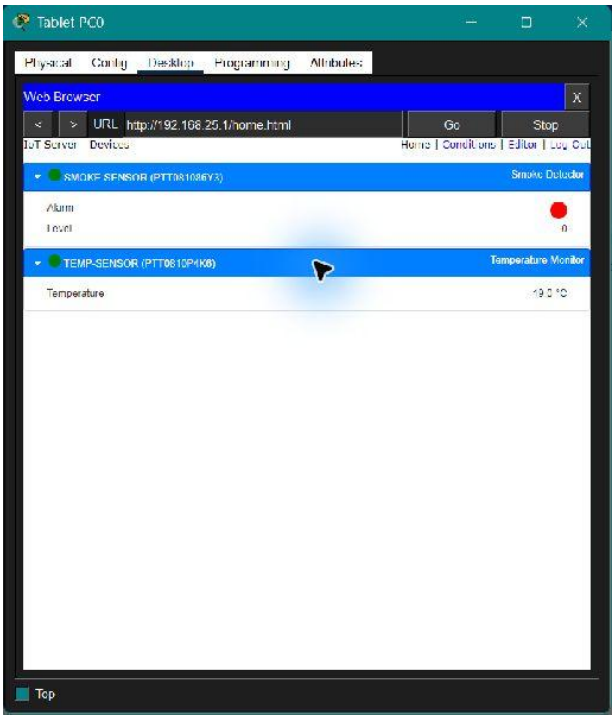
AURA delivered more critical messages on time than fixed SR in the moderate and high-rate conditions. The tight-deadline condition remained difficult for every mode, and baseline exceeded AURA there. The feasibility check reduced high-rate retry cost in the development set, but did not show a DDR gain; on fresh seeds, full AURA and the version without this check both reached 90.4% DDR. These are descriptive results for three short synthetic workloads.

Cisco Packet Tracer implementation

The editable packet_tracer/topology/AURA-UDP-campus.pkt models the campus side of the project. PC0 on the sensor LAN reaches Server0 at 10.10.30.10 through Router0 and Router1. Separately, a native Temperature Monitor and Smoke Detector register with HomeGateway0; Tablet PC0 displays them at 192.168.25.1.



PC0 browser: routed HTTP page served by Server0.



Tablet gateway view: smoke and temperature devices registered.

Figures 2 and 3 Original 513-pixel Packet Tracer captures; captions identify the verified checks

Addressing and the two verified paths

Segment	Addresses	Role
Sensor LAN	10.10.10.0/24	PC0–PC2 and HomeGateway0 reach Router0.
Router transit	10.10.20.0/30	Router0 and Router1 exchange routed traffic.
Server LAN	10.10.30.0/24	Server0 is 10.10.30.10.
Gateway IoT LAN	192.168.25.0/24	Tablet and native sensors use the gateway's local network.

PC0's HTTP check crosses both routers to Server0. The separate tablet check stays on the gateway LAN and shows TEMP-SENSOR and SMOKE-SENSOR. PC3 can also load the page, but shares Server0's LAN, so that check alone does not prove routing.

Packet Tracer does not run AURA or feed NS2. A triggered smoke transition remains unverified; the gateway settings are for a lab demonstration.

Results pipeline and reviewer interface

A simulation run is useful only if its output can be inspected and counted consistently. The repository keeps the run configuration, the raw event sequence, per-run measurements, and summary tables. This allows a reviewer to trace a plotted value back to generated messages rather than accepting a dashboard number without evidence.

Stage	Main file or output	What happens
1 Simulate	ns2/run.tcl	Builds the network and schedules data, ACKs, loss, deadlines, and competing CBR traffic.
2 Record	events.csv, control.csv, background.csv	Logs message events, sender policy decisions, and received competing traffic.
3 Calculate	python/analysis/metrics.py	Counts unique deliveries, on-time critical arrivals, delay, retry cost, and goodput.
4 Repeat	scripts/multiseed.py, scripts/ablation.py	Runs the mode comparison and four policy variants for paired seed indices.
5 Present	summary.csv, SVG plots, dashboard	Shows the measured comparisons and lets a reviewer inspect saved message events.

Why the measurements are auditable

Each message has an ID, so duplicate packet arrivals do not become multiple successful deliveries. A critical message that arrives after its deadline is still a delivery for PDR but is a DDR miss. Lost, expired, and sender-abandoned alerts remain in DDR's denominator. The competing flow is measured at its receiving agent rather than inferred from offered load. These choices prevent a large retry count from being mistaken for useful alert delivery.

The experiment configuration is checked in beside the CSVs. Separate RNG streams model data and ACK errors. A matched seed index gives a consistent comparison framework, but different sending policies consume different random draws; it does not guarantee identical per-packet losses. The result tables report means and paired bootstrap intervals for the chosen scenarios, not a universal performance guarantee.

What the dashboard does

The local Python dashboard is a guided viewer for the saved NS2 experiment. A reviewer can choose a condition, compare baseline, fixed Selective Repeat, and AURA, inspect the feasibility-rule ablation, and follow one message through generation, sending, retry, delivery, or deadline miss. Its files live in python/dashboard/. It reads committed results; opening the page neither starts a simulator nor connects to Packet Tracer.

Code and build checks

The validation record reports 19 passing Python and NS2 checks. They cover deadline expiration, abandonment, bounded-window response, the DDR denominator, competing-flow goodput, and chart generation. GitHub Actions installs NS2, runs the tests, checks dashboard syntax and data paths, and executes small comparison and ablation smoke sweeps on pushes. Full 30-seed sweeps were also repeated and their key CSV outputs matched byte for byte.

Review 2 completion assessment

The present project is a working research prototype with an editable campus demonstration. The 70–75% estimate reflects completion of the main implementation and evidence pipeline, while allowing substantial work for stronger validation and the final demonstration. It is an assessment of deliverables, not a score assigned by the course or a percentage derived from lines of code.

Work area	Evidence already available	Work still needed
NS2 transport	Three sender modes, ACK handling, deadlines, RTT-based timing, and a bounded window execute in ns2/run.tcl.	Refine the forward-time estimate; test behavior over wider queueing and congestion regimes.
Experimental analysis	1,260 recorded runs, per-run data, summaries, plots, policy ablations, and fresh-seed replication.	Use longer and more varied workloads; study multiple competitors and fairness more directly.
Packet Tracer campus	Saved .pkt topology, routed PC0 HTTP, ping checks, native sensor registration, and gateway screenshots.	Verify a triggered smoke transition and capture a clearer end-to-end demonstration sequence.
Dashboard and documentation	Local results viewer, README, protocol description, build guide, validation record, and this report.	Polish final presentation and ensure a reviewer can reproduce the demonstration without setup surprises.
Final claim	Measured gains in two scenarios and a documented failure case; feasibility ablation is reported honestly.	Decide whether an improved decision rule gives a repeatable DDR benefit without harming routine traffic.

What the current results support

- AURA's complete sender policy improved on-time critical delivery against fixed SR in the moderate-loss and high-rate development scenarios.
- The tight-deadline setting exposed a real limitation: few critical alerts reached the receiver on time, and baseline did better than AURA.
- The newer feasibility decision saved some retry traffic in the development high-rate case, but no DDR improvement was demonstrated; fresh-seed high-rate DDR was equal with and without it.

What should not be claimed yet

The data do not establish that AURA is generally superior across networks or that its small adaptive window is Internet-grade congestion control. Packet Tracer shows connectivity and IoT registration, not AURA transport behavior. The native sensors in Packet Tracer are not live inputs to the NS2 workload. Keeping those boundaries explicit makes the final review stronger because every claim has a matching check.

Verification and final review plan

The repository records 19 passing Python and NS2 checks, repeatable sweep outputs, and a GitHub Actions workflow that runs tests and small NS2 smoke sweeps. The saved Packet Tracer checks cover routed HTTP, pings, and native IoT registration. Versioned Git commits and releases provide rollback points.

Remaining work for the final review

- Test longer and more varied workloads, more competing traffic, and a wider range of deadlines and loss conditions.
- Improve the forward-time estimate with queueing uncertainty and evaluate whether the deadline decision improves DDR without starving routine traffic.
- Strengthen congestion and fairness evaluation; the current bounded window is a lab heuristic, not an Internet-grade controller.
- Capture a verified smoke-event transition in Packet Tracer and prepare a concise live demonstration and final report.

How to reproduce the evidence

Open packet_tracer/topology/AURA-UDP-campus.pkt in Packet Tracer 9.0. On PC0, browse to http://10.10.30.10. On Tablet PC0, browse to http://192.168.25.1 and inspect the gateway device list.

In Ubuntu or WSL with NS2 2.35 installed, run python3 scripts/multiseed.py and python3 scripts/ablation.py from the repository root. Open experiments/plots/multiseed.svg and docs/RESEARCH_EVALUATION.md for the measured comparison. To inspect the local dashboard, start python3 -m http.server 8000 --bind 127.0.0.1 from the repository root, then open http://localhost:8000/python/dashboard/.

Source files in the repository

File or folder	What it contains
ns2/run.tcl	Topology, transport behavior, deadlines, ACKs, and logs.
scripts/ and python/analysis/	Experiment runners, metrics, and plots.
experiments/results/	Per-run and summary data for development and fresh seeds.
packet_tracer/	Editable campus topology, web page, build guide, and captures.
docs/PROTOCOL.md and docs/VALIDATION.md	Decision rules and verification record.

Review 2 conclusion: the core simulation, comparison pipeline, campus model, and presentation evidence are implemented. The final phase should focus on broader validation and a clearer demonstration of when the deadline-aware decision helps or fails.